

网络安全

Violet

2026-04-06

Chapter 0

目 录

I	密码学基础	
1.	密码学概论	5
2.	哈希函数与消息认证	6
2.1.	哈希函数概述	6
2.1.1.	核心特性	6
2.1.2.	辨析：单向哈希 vs 双向加密	6
2.2.	主流哈希算法	7
2.2.1.	简要发展历程	7
2.2.2.	MD5	7
2.2.3.	SHA-1	7
2.2.4.	SHA-2	7
2.2.5.	SHA-3 (Keccak)	7
2.2.6.	SM3	7
2.3.	哈希算法的安全性分析	8
2.3.1.	三类基础安全性	8
2.3.2.	碰撞攻击	9
2.3.3.	长度扩展与结构性风险	9
2.4.	哈希算法的应用	9
2.4.1.	推荐应用场景	9
2.4.2.	禁止或不推荐场景	9
2.4.3.	彩虹表、弃用清单与迁移策略	9
2.4.4.	加盐哈希与 KDF	10
2.4.5.	其他重要实践	10
3.	对称加密	11
4.	非对称加密	12
5.	数字签名	13



密码学基础

- 1. 密码学概论 ····· 5
- 2. 哈希函数与消息认证 ····· 6
 - 2.1. 哈希函数概述 ····· 6
 - 2.2. 主流哈希算法 ····· 7
 - 2.3. 哈希算法的安全性分析 ····· 8
 - 2.4. 哈希算法的应用 ····· 9
- 3. 对称加密 ····· 11
- 4. 非对称加密 ····· 12
- 5. 数字签名 ····· 13

Chapter 1

密码学概论

Chapter 2

哈希函数与消息认证

1 哈希函数概述

哈希函数（Hash Function）是将任意长度输入映射为固定长度摘要的函数，常记为：

$$h = H(m)$$

其中， m 是消息， h 是摘要（digest）。在密码学场景中，哈希函数通常强调 **不可逆** 与 **抗攻击**，不仅仅是“算得快”。

1.1 核心特性

密码学哈希函数的核心安全目标包括：

- 确定性：同一输入总是得到同一输出
- 高效性：可在可接受时间内完成计算
- 抗原像（Preimage Resistance）：给定 h ，难以找到 m 使得 $H(m) = h$
- 抗第二原像（Second-Preimage Resistance）：给定 m_1 ，难以找到 $m_2 \neq m_1$ 且 $H(m_2) = H(m_1)$
- 抗碰撞（Collision Resistance）：难以找到任意 $m_1 \neq m_2$ 使得 $H(m_1) = H(m_2)$
- 雪崩效应：输入微小变化会导致输出显著变化

NOTE

“抗碰撞”最强、成本也最高。工程中常用“位数足够长 + 选用未被攻破算法”来降低风险。

1.2 辨析：单向哈希 vs 双向加密

维度	单向哈希	双向加密
是否可逆	不可逆（理论上无法还原明文）	可逆（持有密钥可解密）
输入输出	任意长度 -> 固定长度摘要	明文 <-> 密文（长度通常相关）
核心目的	完整性校验、指纹、签名前处理	保密传输与存储
密钥需求	一般不需要密钥（HMAC 例外）	必须有密钥
典型算法	SHA-256 / SHA-3 / SM3	AES / ChaCha20 / SM4

WARNING

常见误区：把哈希当“加密”使用。哈希不能解密，也不能直接提供机密性。

2 主流哈希算法

2.1 简要发展历程

- 1990s: MD4/MD5 流行，后续逐步发现碰撞风险
- 1995: SHA-1 发布，曾长期广泛部署
- 2001: SHA-2 (SHA-224/256/384/512) 发布，成为主流
- 2012: Keccak 竞赛胜出，标准化为 SHA-3
- 2010s: SM3 在国内标准体系中广泛采用

2.2 MD5

- 摘要长度: 128 bit
- 设计背景: Rivest 于 1992 年提出，基于 Merkle-Damgard 结构
- 现状: 已被实用碰撞攻击击破，不可用于安全相关签名或证书

典型风险: 攻击者可构造两份不同文件得到相同 MD5，导致“指纹一致但内容不同”。

2.3 SHA-1

- 摘要长度: 160 bit
- 历史地位: 曾用于 TLS、代码签名、Git 对象标识等
- 现状: 已出现实际可行碰撞，安全场景应弃用

2.4 SHA-2

- 家族: SHA-224 / SHA-256 / SHA-384 / SHA-512
- 结构: 迭代压缩 (Merkle-Damgard 系)
- 现状: 截至目前仍是工程主力，SHA-256/512 使用广泛

TIP

一般业务优先选 SHA-256；对高性能 64 位平台可考虑 SHA-512/256 等变体。

2.5 SHA-3 (Keccak)

- 摘要长度: 常见 224/256/384/512 bit
- 核心结构: Sponge (海绵) 结构，区别于传统 Merkle-Damgard
- 优势: 设计路径独立于 SHA-2；天然支持 XOF (如 SHAKE)

NOTE

SHA-3 不是“替代 SHA-2 的紧急补丁”，更像是并行体系，为算法多样性与长期稳健性提供保障。

2.6 SM3

- 摘要长度: 256 bit
- 标准背景: 我国商用密码哈希标准，常见于国密体系
- 应用生态: 与 SM2 (签名/密钥交换) 和 SM4 (分组加密) 配套使用

SM3 发展脉络 (简)

- 2000s: 国内开始推进自主密码算法体系

- 2010：发布 SM3 相关国家/行业标准
- 后续：在政务、金融、关键基础设施等场景持续落地

SM3 详细算法解析

SM3 采用分组迭代方式，整体流程可拆为 5 步：

1. 消息填充：先补 1，再补若干 0，最后附加消息长度，使总长度满足分组要求
 2. 分组：按 512 bit 划分消息块
 3. 消息扩展：每块扩展为 $W[0..67]$ 和 $W'[0..63]$
 4. 压缩迭代：维护 8 个 32 位工作寄存器并进行 64 轮更新
 5. 输出：与上一链接变量异或得到新的链接变量，最终输出 256 bit 摘要
- 常见符号关系（示意）如下：

$$W'[j] = W[j] \oplus W[j + 4]$$

$$V_{i+1} = CF(V_i, B_i)$$

其中， B_i 为第 i 个消息分组， CF 为压缩函数。

Algorithm

SM3 压缩函数（理解版）：

- 输入：上一轮链接变量 V_i 、当前消息块 B_i
- 执行：
 - 进行消息扩展，得到 W 与 W'
 - 重复 64 轮布尔运算、模加、循环左移
 - 更新 A..H 八个寄存器
- 输出： $V_{i+1} = V_i \oplus (A \parallel B \parallel C \parallel D \parallel E \parallel F \parallel G \parallel H)$

TIP

学习“详细算法”时，建议按“填充 -> 扩展 -> 轮函数 -> 链接变量更新”四层结构记忆，而不是死背常量表。

3 哈希算法的安全性分析

3.1 三类基础安全性

性质	攻击目标	理想攻击复杂度
抗原像	给定 h 找 m	约 2^n
抗第二原像	给定 m_1 找 m_2	约 2^n
抗碰撞	找任意 $m_1 \neq m_2$	约 $2^{(n/2)}$ （生日界）

若哈希输出长度为 n 位，则碰撞攻击理论下界约为：

$$O(2^{\frac{n}{2}})$$

这就是为何摘要长度不能太短。

3.2 碰撞攻击

常见碰撞攻击路径：

- 随机碰撞搜索：依赖生日悖论，复杂度约 $2^{\frac{n}{2}}$
- 构造型碰撞：利用算法结构缺陷，远低于理论复杂度
- 选择前缀碰撞：攻击者可控制两段前缀并构造同摘要后缀

工程影响：

- 伪造“看似相同指纹”的文件对
- 干扰签名/证书信任链（若系统仍依赖弱哈希）
- 造成审计与取证混淆

3.3 长度扩展与结构性风险

对某些 Merkle-Damgard 结构算法，若协议设计不当，可能出现长度扩展攻击：

- 错误用法： `Hash(key || msg)` 直接当作认证码
- 正确做法：使用 HMAC（如 HMAC-SHA-256 / HMAC-SM3）

WARNING

“哈希 + 拼接密钥”不是通用安全方案。消息认证请优先使用标准 MAC 构造。

4 哈希算法的应用

4.1 推荐应用场景

- 完整性校验：下载文件校验、镜像校验、区块数据校验
- 数字签名前处理：先哈希再签名，提升效率并统一输入长度
- 消息认证：基于 HMAC 的 API 签名、防篡改
- 内容寻址：对象存储去重、版本指纹（注意碰撞与迁移策略）

4.2 禁止或不推荐场景

DANGER

- 使用 MD5/SHA-1 作为安全签名依据
- 明文密码直接做 `SHA-256(password)` 后存库
- 把哈希当加密来“保护敏感数据”
- 自研 MAC、自定义“盐+哈希+截断”协议替代标准方案

4.3 彩虹表、弃用清单与迁移策略

INFO

彩虹表是“预计算哈希反查表”，可显著降低弱口令破解成本。没有盐或盐过短时风险更高。

常见弃用清单（安全场景）：

- MD5（全面弃用）

- SHA-1（全面弃用）
- 无盐快速哈希存密码（全面弃用）
- 短摘要截断（如随意截成 32 64 bit）用于抗碰撞场景

迁移原则：

1. 新增数据全部切换到强算法
2. 老数据采用“登录时重哈希”或批量迁移
3. 保留算法版本字段，支持平滑升级

4.4 加盐哈希与 KDF

口令存储的基本原则不是“快”，而是 故意变慢 + 增加内存成本。

推荐策略：

- 每个用户独立随机盐（Salt）
- 可选服务端 Pepper（存放在 HSM/KMS 或独立配置）
- 使用专用口令 KDF，而非通用快速哈希

方案	特点	适用性
PBKDF2	迭代多次，成熟稳定	兼容性好，但抗 GPU 相对一般
bcrypt	内置盐，成本参数可调	广泛可用，输出长度有限
scrypt	强调内存硬性	抗并行硬件较好
Argon2id	当前口令哈希首选之一	综合抗侧信道与抗 GPU 能力较好

典型口令存储形式：

`$算法$参数$盐$派生值`

例如可记录：算法版本、时间成本、内存成本、并行度，便于后续升级。

4.5 其他重要实践

- 域分离：不同业务用途使用不同前缀/标签，避免跨协议复用风险
- 规范化：签名前先统一编码与字段顺序，避免“同义不同串”
- 常量时间比较：比较摘要/标签时避免时间侧信道
- 密钥生命周期管理：MAC 密钥需轮换、分级、可审计

TIP

工程上最稳妥的策略是：

- 完整性：SHA-256 / SHA-3 / SM3
- 认证：HMAC-SHA-256 / HMAC-SM3
- 口令：Argon2id（优先）或 bcrypt / scrypt / PBKDF2

Chapter 3

对称加密

Chapter 4

非对称加密

Chapter 5

数字签名
